

Ch04 데이터 프레임(Data Frame) 자료 다루기

(Ch16과 Ch04 내용을 함께 구성한다.)

```
#DataFrame() #loc() #iloc() #set_index() #reset_index()

#iterrows() #at() #drop() #rename()

#value_counts() #sort_values() #rename() #head() #tail() #info() #describe()

#read_excel() #to_excel() #read_csv() #to_csv()
```

외장 자료형: Pandas 자료형

- > pandas라는 패키지를 import 해서 사용 가능한 추가 자료 구조
- > Data Type : Series, Dataframe
- > 관련 함수 : Series(), DataFrame()

In []:

Pandas 자료형: 시리즈(Series)

- > **1차원 배열**로 여러 값을 나열한 자료 구조
- > 각 데이터 항목은 index로 식별한다.
- > 데이터 프레임의 데이터를 구성하는 열 값들을 시리즈로 추출하여 조작할 수 있다.
- > pandas 패키지를 통해 사용할 수 있다.

In [2]:

```
## 시리즈(Series) 형: 생성, index 강제 부여
import pandas as pd

a = pd.Series([11, 22, 33, 44], index = [1, 2, 3, 4]) #index 강제 부여
a
```

Out[2]:

```
1    11
2    22
3    33
4    44
dtype: int64
```

In [3]:

```
## 시리즈(Series) 형: 생성 index 자동 부여
import pandas as pd

a = pd.Series([11, 22, 33, 44]) #index 자동 부여
a
```

```
Out[3]: 0    11
        1    22
        2    33
        3    44
        dtype: int64
```

```
In [11]: ## 시리즈(Series) 형: 사용
print(a[0])
print(a[1:])
```

```
11
1    22
2    33
3    44
dtype: int64
```

```
In [4]: ## 시리즈(Series) 형: 사용
print(a[:])
print(a[1:-1])
```

```
0    11
1    22
2    33
3    44
dtype: int64
1    22
2    33
dtype: int64
```

```
In [ ]:
```

Pandas 자료형: 데이터 프레임(Data Frame)

- > 행과 열로 나열되는 **2차원적** 자료 구조
- > 행과 열 값들은 딕셔너리 구조를 활용하여 초기화 한다.
- > 하나의 열 값들은 시리즈형으로 추출하여 사용 가능하다.
- > 복수 개의 열 값들은 데이터 프레임형으로 추출하여 사용 가능하다.
- > pandas 패키지를 통해 사용할 수 있다.

데이터 프레임 생성

열 생성: 열명(**열 변수**)에 대응되는 값들을 리스트로 선언

행 이름 지정: 열의 행에 대응하는 행 명 index를 지정(생략하면 0, 1, 2,...)

```
In [5]: ## [Data Frame] 생성
import pandas as pd

df = pd.DataFrame({'Class' : [1, 1, 2],      #열 이름이 열 index 역할
                   'Eng'   : [87, 79, 80],
                   'Mat'   : [80, 90, 80]},
                  index = ['Kims', 'Lees', 'Parks']) #행 index 강제 부여

df
```

```
Out[5]:
```

	Class	Eng	Mat
Kims	1	87	80
Lees	1	79	90
Parks	2	80	80

```
In [6]: ## [Data Frame] 생성
import pandas as pd

df = pd.DataFrame({'Class' : [1, 1, 2],      # 열 이름이 열 index 역할
                  'Eng' : [87, 79, 80],
                  'Mat' : [80, 90, 80]}) # 행 index 자동 부여

df
```

```
Out[6]:
```

	Class	Eng	Mat
0	1	87	80
1	1	79	90
2	2	80	80

데이터 프레임 데이터 추출

> 열 변수를 사용한 열의 행 추출

```
In [7]: ## [Data Frame] 데이터 추출 : 열 변수로 열 추출
df['Eng']
```

```
Out[7]:
```

0	87
1	79
2	80

Name: Eng, dtype: int64

```
In [8]: ## [Data Frame] 데이터 추출 : 열 변수 나열로 열 추출
df[['Eng', 'Mat']]      # 'Eng'와 'Mat' 열 값들을 Data Frame으로 추출
```

```
Out[8]:
```

	Eng	Mat
0	87	80
1	79	90
2	80	80

```
In [20]: ## [Data Frame] 데이터 추출 : 원하는 열의 행 추출
collist = ['Eng', 'Mat']
x = df[collist];          # 'Eng'와 'Mat'는 List 형태로 입력
x
```

```
Out[20]:
```

	Eng	Mat
0	87	80
1	79	90
2	80	80

```
In [12]: ## [Data Frame] 데이터 추출 : 열 변수로 열 추출
print(type(df['Eng']))
print(type(df[['Eng', 'Mat']]))
```

```
<class 'pandas.core.series.Series'>
<class 'pandas.core.frame.DataFrame'>
```

>열을 추출하여 조건식을 사용한 행 추출

```
In [13]: ## [Data Frame] 데이터 생성
import pandas as pd

df = pd.DataFrame({'Class' : [1, 1, 2],      #열 이름이 열 index 역할
                  'Eng' : [87, 79, 80],
                  'Mat' : [80, 90, 80]}) #행 index 자동 부여

df
```

```
Out[13]:
```

	Class	Eng	Mat
0	1	87	80
1	1	79	90
2	2	80	80

df['Class'] == 1 #Query 문의 행 선택 조건 where ~에 해당

```
In [25]: ## [Data Frame] 데이터 추출 : 조건부 행 검사
df['Class'] == 1 #비교 연산이기 때문에 결과는 True / False
```

```
Out[25]: 0      True
1      True
2     False
Name: Class, dtype: bool
```

```
In [14]: ## [Data Frame] 데이터 추출 : 조건부 행 추출
df[df['Class'] == 1] #df에서 (df['Class'] == 1)가 True인 행만 추출
```

```
Out[14]:
```

	Class	Eng	Mat
0	1	87	80
1	1	79	90

```
In [15]: ## [Data Frame] 데이터 추출 : 조건부 행 추출
x = df[df['Eng'] >= 80];          # 'Eng' 열 값들을 시리즈(Series)로 추출해서 b
x
```

Out[15]:

	Class	Eng	Mat
0	1	87	80
2	2	80	80

```
In [16]: ## [Data Frame] 데이터 추출 : 조건부 행 추출
x = df[(df['Eng'] >= 80) & (df['Mat'] >= 80)];
x
```

Out[16]:

	Class	Eng	Mat
0	1	87	80
2	2	80	80

> 행 추출하여 열 선택 추출

```
In [17]: ## [Data Frame] 데이터 추출: 조건부 행, 열 추출
df[df['Class'] == 1]['Mat'] # df[df['Class'] == 1] 결과는 프레임이므로 df[]는 프
```

Out[17]:

0	80
1	90

Name: Mat, dtype: int64

```
In [18]: ## [Data Frame] 데이터 추출: 조건부 행, 열 추출
ddf = df[df['Class'] == 1] # df[df['Class'] == 1] 결과는 프레임
ddf['Mat'] #프레임의 열 선택
```

Out[18]:

0	80
1	90

Name: Mat, dtype: int64

```
In [19]: ## [Data Frame] 데이터 추출: 조건부 행, 열 추출
df[df['Class'] == 1][['Mat', 'Eng']]
```

Out[19]:

	Mat	Eng
0	80	87
1	90	79

```
In [20]: ## [Data Frame] 데이터 추출: 조건부 행, 열 추출
ddf = df[df['Class'] == 1] # df[df['Class'] == 1] 결과는 프레임
ddf[['Mat', 'Eng']] #프레임의 열 선택
```

Out[20]:

	Mat	Eng
0	80	87
1	90	79

```
In [21]: ## [Data Frame] 데이터 추출: 조건부 행, 열 추출
x = df[(df['Eng'] >= 80) & (df['Mat'] >= 80)][['Eng', 'Mat']]; ### 'Eng', 'Mat'
x
```

```
Out[21]:
```

	Eng	Mat
0	87	80
2	80	80

(알아 두면 좋아요) .을 이용해 변수를 간단하게 추출하기

> DataFrame.Col_variable 로 열 추출 가능

```
In [22]: ## [Data Frame] 데이터 추출 :
df.Mat
```

```
Out[22]: 0    80
1    90
2    80
Name: Mat, dtype: int64
```

```
In [ ]:
```

데이터 추출 : loc[] 이용

> loc[]는 데이터프레임에서 특정 행과 열을 위치 값(location)으로 선택하는 인덱싱(indexing) 방법 중 하나

> Index는 행을 식별하는 Key 역할을 함.

> **Indexing**은 Index를 이용해 데이터를 추출하는 방법 > 결과적으로 **행을 추출**

> loc[**row_index**, **col_index**]로 추출

> loc[x,y] 숫자 index가 x 이상 ~ y 이하 행을 추출

```
In [23]: ## [Data Frame] 생성
import pandas as pd

df = pd.DataFrame({'Class' : [1, 1, 2],
                   'Eng' : [87, 79, 80],
                   'Mat' : [80, 90, 80]},
                  index = ['Kims', 'Lees', 'Parks']) #index 강제 부여
df
```

```
Out[23]:
```

	Class	Eng	Mat
Kims	1	87	80
Lees	1	79	90
Parks	2	80	80

```
In [24]: ## [Data Frame] 데이터 추출: loc[] > 행, 열 추출
# loc[row_index, col_index]
result = df.loc['Kims', 'Eng'] #결과는 Series 자료형
print(type(result))
result
```

```
<class 'numpy.int64'>
```

Out[24]: np.int64(87)

```
In [25]: ## [Data Frame] 데이터 추출: loc[] > 행 (Series 자료형) 추출
# loc[row_index, col_index]
result = df.loc['Kims'] # 결과는 Series 자료형
print(type(result))
result
```

<class 'pandas.core.series.Series'>

Out[25]: Class 1
Eng 87
Mat 80
Name: Kims, dtype: int64

```
In [29]: ## [Data Frame] 데이터 추출: loc[] > 행 (Data Frame 자료형) 추출
result = df.loc[['Kims', 'Lees']] # 결과는 Data Frame 자료형
print(type(result))
result
```

<class 'pandas.core.frame.DataFrame'>

Out[29]:

	Class	Eng	Mat
Kims	1	87	80
Lees	1	79	90

```
In [26]: ## [Data Frame] 데이터 추출: loc[] > 행 (Data Frame 자료형) 추출
result = df.loc[['Kims']] # 결과는 Data Frame 자료형
print(type(result))
result
```

<class 'pandas.core.frame.DataFrame'>

Out[26]:

	Class	Eng	Mat
Kims	1	87	80

```
In [28]: ## [Data Frame] 데이터 추출: loc[] > 복수 행 추출
result = df.loc[: ] # 전체 행 선택, 결과는 Data Frame 자료형
result
```

Out[28]:

	Class	Eng	Mat
Kims	1	87	80
Lees	1	79	90
Parks	2	80	80

```
In [30]: ## [Data Frame] 데이터 추출: loc[] > 행 (Data Frame 자료형) 추출
result = df.loc['Kims':'Lees'] # 결과는 Data Frame 자료형
print(type(result))
result
```

<class 'pandas.core.frame.DataFrame'>

```
Out[30]:
```

	Class	Eng	Mat
Kims	1	87	80
Lees	1	79	90

> 행 추출하여 열 선택 추출

>> loc[*row_index*, *col_index*]로 추출

```
In [32]: ## [Data Frame] 데이터 추출: loc[] > 복수 행, 선택 열 추출
# loc[row_index, col_index]
result = df.loc[['Kims', 'Lees'], 'Mat'] #결과는 Series 자료형
result = df.loc[['Kims', 'Lees'], ['Mat']] #결과는 Data Frame 자료형
result
```

```
Out[32]:
```

	Mat
Kims	80
Lees	90

```
In [33]: ## [Data Frame] 데이터 추출: loc[] > 복수 행, 선택 열 추출
result = df.loc[['Kims', 'Lees'], :] #전체 행 선택, #결과는 Data Frame 자료형
result = df.loc[['Kims', 'Lees'], ['Mat', 'Eng']] #결과는 Data Frame 자료형
result
```

```
Out[33]:
```

	Mat	Eng
Kims	80	87
Lees	90	79

> loc[조건식] 사용 추출

```
In [34]: ## [Data Frame] 데이터 추출: loc[] > 조건부 추출
result = df.loc[df['Eng'] >= 80] #[비교 연산식]이 True인 행 선택
result
```

```
Out[34]:
```

	Class	Eng	Mat
Kims	1	87	80
Parks	2	80	80

Index를 이용한 추출

```
In [35]: ## [Data Frame] 생성
import pandas as pd

df = pd.DataFrame({ 'Name' : ['Kims', 'Lees', 'Parks'],
                    'Class' : [1, 1, 2],
                    'Eng' : [87, 79, 80],
                    'Mat' : [80, 90, 80]}) #index 자동 부여

df
```


Out[35]:

	Name	Class	Eng	Mat
--	------	-------	-----	-----

0	Kims	1	87	80
---	------	---	----	----

1	Lees	1	79	90
---	------	---	----	----

2	Parks	2	80	80
---	-------	---	----	----

In [36]: `## [Data Frame] 데이터 추출: index로 추출`
`df.loc[0]` #0번 index 행 추출

Out[36]:

Name	Kims
Class	1
Eng	87
Mat	80

Name: 0, dtype: object

In [37]: `## [Data Frame] 데이터 추출: index로 추출`
`df.loc[[0, 2]]` #0, 2번 index 행 추출

Out[37]:

	Name	Class	Eng	Mat
--	------	-------	-----	-----

0	Kims	1	87	80
---	------	---	----	----

2	Parks	2	80	80
---	-------	---	----	----

In [38]: `## [Data Frame] 데이터 추출: index로 추출`
`df.loc[0:2]` #0~2번 index 행 추출

Out[38]:

	Name	Class	Eng	Mat
--	------	-------	-----	-----

0	Kims	1	87	80
---	------	---	----	----

1	Lees	1	79	90
---	------	---	----	----

2	Parks	2	80	80
---	-------	---	----	----

In [39]: `## [Data Frame] 데이터 추출: index로 추출`
`df.loc[1:]` #1에서 끝까지 index 행 추출

Out[39]:

	Name	Class	Eng	Mat
--	------	-------	-----	-----

1	Lees	1	79	90
---	------	---	----	----

2	Parks	2	80	80
---	-------	---	----	----

> 행 추출하여 열 선택 추출

In [40]: `## [Data Frame] 데이터 추출: index로 추출`
`df.loc[:,1, ['Eng', 'Mat']]` #1에서 끝까지 index 행 추출

Out[40]:

	Eng	Mat
--	-----	-----

0	87	80
---	----	----

1	79	90
---	----	----

```
In [41]: ## [Data Frame] 생성
import pandas as pd

df = pd.DataFrame({'Class' : [1, 1, 2],
                   'Eng' : [87, 79, 80],
                   'Mat' : [80, 90, 80]},
                  index = ['Kims', 'Lees', 'Parks']) #index 강제 부여
#
df            index = [1, 2, 3]) #index 강제 부여
```

```
Out[41]:
```

	Class	Eng	Mat
Kims	1	87	80
Lees	1	79	90
Parks	2	80	80

숫자 index 번호가 없을 때, index 번호로 .loc[:] 사용 불가능

```
In [42]: ## [Data Frame] 데이터 추출:
## index 번호가 없을 때, index 번호로 .loc[:] 사용 불가능
df.loc[0:1]
```

TypeError

Traceback (most recent call last)

Cell In[42], line 3

```
1 ## [Data Frame] 데이터 추출:  
2 ## index 번호가 없을 때, index 번호로 .loc[:] 사용 불가능  
----> 3 df.loc[0:1]
```

File ~\anaconda3\Lib\site-packages\pandas\core\indexing.py:1191, in [_LocationIndexer._getitem__\(self, key\)](#)

```
1189 maybe_callable = com.apply_if_callable(key, self.obj)  
1190 maybe_callable = self._check_deprecated_callable_usage(key, maybe_callable)  
-> 1191 return self._getitem_axis(maybe_callable, axis=axis)
```

File ~\anaconda3\Lib\site-packages\pandas\core\indexing.py:1411, in [_iLocIndexer._getitem_axis\(self, key, axis\)](#)

```
1409 if isinstance(key, slice):  
1410     self._validate_key(key, axis)  
-> 1411     return self._get_slice_axis(key, axis=axis)  
1412 elif com.is_bool_indexer(key):  
1413     return self._get_bool_axis(key, axis=axis)
```

File ~\anaconda3\Lib\site-packages\pandas\core\indexing.py:1443, in [_iLocIndexer._get_slice_axis\(self, slice_obj, axis\)](#)

```
1440     return obj.copy(deep=False)  
1442 labels = obj._get_axis(axis)  
-> 1443 indexer = labels.slice_indexer(slice_obj.start, slice_obj.stop, slice_obj.step)  
1445 if isinstance(indexer, slice):  
1446     return self.obj._slice(indexer, axis=axis)
```

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:6662, in [Index.slice_indexer\(self, start, end, step\)](#)

```
6618 def slice_indexer(  
6619     self,  
6620     start: Hashable | None = None,  
6621     end: Hashable | None = None,  
6622     step: int | None = None,  
6623 ) -> slice:  
6624     """  
6625     Compute the slice indexer for input labels and step.  
6626     (...)  
6660     slice(1, 3, None)  
6661     """  
-> 6662     start_slice, end_slice = self.slice_locs(start, end, step=step)  
6664     # return a slice  
6665     if not is_scalar(start_slice):
```

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:6879, in [Index.slice_locs\(self, start, end, step\)](#)

```
6877 start_slice = None  
6878 if start is not None:  
-> 6879     start_slice = self.get_slice_bound(start, "left")  
6880 if start_slice is None:  
6881     start_slice = 0
```

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:6794, in [Index.get_slice_bound\(self, label, side\)](#)

```
6790 original_label = label
```

```

6792 # For datetime indices label may be a string that has to be converted
6793 # to datetime boundary according to its resolution.
-> 6794 label = self._maybe_cast_slice_bound(label, side)
6796 # we need to look up the label
6797 try:

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:6727, in Index._maybe_cast_slice_bound(self, label, side)
6725 # reject them, if index does not contain label
6726 if (is_float(label) or is_integer(label)) and label not in self:
-> 6727     self._raise_invalid_indexer("slice", label)
6729 return label

File ~\anaconda3\Lib\site-packages\pandas\core\indexes\base.py:4301, in Index._raise_invalid_indexer(self, form, key, reraise)
4299 if reraise is not lib.no_default:
4300     raise TypeError(msg) from reraise
-> 4301 raise TypeError(msg)

TypeError: cannot do slice indexing on Index with these indexers [0] of type int

```

iloc[] : 숫자 index 번호가 없을 때, index 번호로 .iloc[:] 사용 가능

```

In [28]: ## [Data Frame] 데이터 추출:
## index 번호가 없을 때, index 번호로 .iloc[:] 사용 가능
df.iloc[0:1] #df.loc[1:1]과 동일

```

```

Out[28]:
   Class  Eng  Mat
Kims      1   87   80

```

In []:

데이터 추출 : iloc[] 이용

> iloc[]는 데이터 추출 전에 **행과 열에 임시로 숫자 index를 지정**하여 사용하는 인덱싱 방법

> loc[]와 달리 조건부 추출이 불가능

> iloc[x:y] **x 이상 ~ y 미만** 행을 추출 >> loc[x:y] 숫자 index가 **x 이상 ~ y 이하** 행을 추출

```

In [43]: ## [Data Frame] 생성
import pandas as pd

df = pd.DataFrame({ 'Name' : ['Kims', 'Lees', 'Parks'],
                    'Class' : [1, 1, 2],
                    'Eng' : [87, 79, 80],
                    'Mat' : [80, 90, 80]}) #index 자동 부여

df

```

```
Out[43]:
```

	Name	Class	Eng	Mat
0	Kims	1	87	80
1	Lees	1	79	90
2	Parks	2	80	80

```
In [44]: ## [Data Frame] 데이터 추출: iloc[] > 행 추출
result = df.iloc[0]      #1개 데이터이므로 Series 형으로 추출
result = df.iloc[[0]]    #Data Frame 형으로 추출
result = df.iloc[:,]     #복수개 데이터이므로 Data Frame 형으로 추출
result = df.iloc[1:3]    #복수개 데이터이므로 Data Frame 형으로 추출
result
```

```
Out[44]:
```

	Name	Class	Eng	Mat
1	Lees	1	79	90
2	Parks	2	80	80

```
In [45]: ## [Data Frame] 데이터 추출: iloc[] > 행 추출
result = df.iloc[[0, 2]] #복수개 데이터이므로 Data Frame 형으로 추출
result
```

```
Out[45]:
```

	Name	Class	Eng	Mat
0	Kims	1	87	80
2	Parks	2	80	80

Index 번호로 열 추출

```
In [46]: ## [Data Frame] 데이터 추출: iloc[] > 행, 열 추출
result = df.iloc[[0, 2], [1]] #열 선택도 열 순서 번호(index)로 명시
result = df.iloc[[0, 2], [1, 0]] #열 선택도 열 순서 번호(index)로 명시
result
```

```
Out[46]:
```

	Class	Name
0	1	Kims
2	2	Parks

> iloc[]는 조건 추출이 불가능

```
In [47]: df.iloc[:, [1]]
```

```
Out[47]:
```

	Class
0	1
1	1
2	2

```
In [61]: df.loc[df['Class'] == 1]
```

```
Out[61]:
```

	Name	Class	Eng	Mat
0	Kims	1	87	80
1	Lees	1	79	90

```
In [62]: df.iloc[df['Class'] == 1]
```

```
-----
NotImplementedError                                Traceback (most recent call last)
~\AppData\Local\Temp\ipykernel_31072\1661649294.py in <module>
----> 1 df.iloc[df['Class'] == 1]

~\anaconda3\lib\site-packages\pandas\core\indexing.py in __getitem__(self, key)
    1151
    1152         maybe_callable = com.apply_if_callable(key, self.obj)
-> 1153         return self._getitem_axis(maybe_callable, axis=axis)
    1154
    1155     def _is_scalar_access(self, key: tuple):

~\anaconda3\lib\site-packages\pandas\core\indexing.py in _getitem_axis(self, key,
axis)
    1698
    1699         if com.is_bool_indexer(key):
-> 1700             self._validate_key(key, axis)
    1701             return self._getbool_axis(key, axis=axis)
    1702

~\anaconda3\lib\site-packages\pandas\core\indexing.py in _validate_key(self, key,
axis)
    1538         if hasattr(key, "index") and isinstance(key.index, Index):
    1539             if key.index.inferred_type == "integer":
-> 1540                 raise NotImplementedError(
    1541                     "iLocation based boolean "
    1542                     "indexing on an integer type "

NotImplementedError: iLocation based boolean indexing on an integer type is not a
vailable
```

> df.iloc[df['Class'] == 1] 해결 방법

```
In [63]: df[df['Class'] == 1]  #df['Class'] == 1인 행 선택하여 데이터 프레임 구성
```

```
Out[63]:
```

	Name	Class	Eng	Mat
0	Kims	1	87	80
1	Lees	1	79	90

```
In [48]: df[df['Class'] == 1].iloc[:, [0, 2, 3]]  #Class=1인 데이터프레임을 구성해서 iloc
```



```
Out[52]:
```

	Name	Class	Eng	Mat
0	Kims	1	87	80
1	Lees	1	79	90
2	Parks	2	80	80

```
In [53]: df.iloc[1:2]
```

```
Out[53]:
```

	Name	Class	Eng	Mat
1	Lees	1	79	90

```
In [ ]:
```

(알아 두면 좋아요) 열 변수를 index로 전환하기

index를 열 변수로 전환도 가능(단, index에 변수명이 있을 때만 가능)

.set_index() : 특정 열을 index로 사용

- > key : 인덱스로 사용하려는 열
- > drop : 인덱스 제거 (default는 True)
- > append : 자동 인덱스 추가 (default는 False), 멀티 인덱스 생성 시 사용
- > inplace : False이면 자신은 안 바꾸고 변경 결과만 반환 (default는 False)

```
In [54]: ## [Data Frame] 생성: index 지정 없이
import pandas as pd

df = pd.DataFrame({'Name' : ['Kims', 'Lees', 'Parks'],
                  'Eng' : [87, 79, 80],
                  'Mat' : [80, 90, 80]},) #index 강제 부여

df
```

```
Out[54]:
```

	Name	Eng	Mat
0	Kims	87	80
1	Lees	79	90
2	Parks	80	80

```
In [55]: # 'Name' 열을 인덱스로 설정: 'Name' 열을 제거하지 않음(drop=False)
ddf = df.set_index('Name') # 'Name' 으로 인덱스 대체

ddf
```


Out[55]:

	Eng	Mat
Name		
Kims	87	80
Lees	79	90
Parks	80	80

```
In [56]: # 'Name' 열을 인덱스로 설정: 'Name' 열을 제거하지 않음(drop=False)
ddf = df.set_index('Name', drop=False) # 'Name' 으로 인덱스 대체
ddf
```

Out[56]:

	Name	Eng	Mat
Name			
Kims	Kims	87	80
Lees	Lees	79	90
Parks	Parks	80	80

```
In [57]: # 'Name' 열을 인덱스로 설정: 'Name' 열을 제거하지 않음(drop=False)
ddf = df.set_index('Name', drop=True) # 'Name' 으로 인덱스 대체
ddf
```

Out[57]:

	Eng	Mat
Name		
Kims	87	80
Lees	79	90
Parks	80	80

```
In [59]: # 'Name' 열을 인덱스로 설정: 기존 'Name' 열은 제거(drop=True)
ddf = df.set_index('Name', drop=True) # 'Name' 으로 인덱스 대체
df      # df는 안 바뀜
```

Out[59]:

	Name	Eng	Mat
0	Kims	87	80
1	Lees	79	90
2	Parks	80	80

inplace=True: 자신을 변경

```
In [60]: # 'Name' 열을 인덱스로 설정: 기존 'Name' 열은 제거(drop=True)
df.set_index('Name', drop=True, inplace=False) # 'Name' 으로 인덱스 대체
df
```

```
Out[60]:
```

	Name	Eng	Mat
0	Kims	87	80
1	Lees	79	90
2	Parks	80	80

```
In [61]: # 'Name' 열을 인덱스로 설정: 기존 'Name' 열은 제거(drop=True)
df.set_index('Name', drop=True, inplace=True) # 'Name'으로 인덱스 대체
df
```

```
Out[61]:
```

	Eng	Mat
Name		
Kims	87	80
Lees	79	90
Parks	80	80

set_index(): 멀티 index 생성(append=True)

```
In [63]: ## [Data Frame] 생성: index 지정 없이
import pandas as pd

df = pd.DataFrame({'Name' : ['Kims', 'Lees', 'Parks'],
                   'Eng' : [87, 79, 80],
                   'Mat' : [80, 90, 80]},) #index 강제 부여
df
```

```
Out[63]:
```

	Name	Eng	Mat
0	Kims	87	80
1	Lees	79	90
2	Parks	80	80

```
In [64]: # 'Name' 열을 인덱스로 설정: 멀티 index 생성(append=True)
ddf = df.set_index('Name', append=True, drop=True, inplace=False) # 'Name'으로 인덱스 대체
ddf
```

```
Out[64]:
```

		Eng	Mat
	Name		
0	Kims	87	80
1	Lees	79	90
2	Parks	80	80

```
In [65]: ## 멀티 index 검색
ddf.loc[0]
```

Out[65]:

	Eng	Mat
Name		
Kims	87	80

```
In [57]: ## 멀티 index 검색
ddf.loc[0, 'Kims'] #두개의 index로 행을 검색
```

Out[57]:

Eng	87
Mat	80

Name: (0, Kims), dtype: int64

.reset_index()

설정된 index를 삭제하고, 기본 숫자 index 부여

- > drop : True이면 제거되는 index를 열로 추가하지 않음(제거 효과) (default는 True)
- > drop : False이면 제거되는 index를 열로 추가
- > inplace : False이면 자신은 안 바꾸고 변경 결과만 반환 (default는 False)

```
In [86]: ## [Data Frame] 생성: index 없이
import pandas as pd

df = pd.DataFrame({ 'Class' : [1, 1, 2],
                    'Eng' : [87, 79, 80],
                    'Mat' : [80, 90, 80]},
                  index = ['Kims', 'Lees', 'Parks']) #index 강제 부여

df
```

Out[86]:

	Class	Eng	Mat
Kims	1	87	80
Lees	1	79	90
Parks	2	80	80

```
In [87]: # 현재 인덱스를 일반 열로 변경: index를 reset하고, 열로 추가(drop=False)
ddf = df.reset_index() #제거되는 index를 열로 추가 (default: drop=False)
ddf
```

Out[87]:

	index	Class	Eng	Mat
0	Kims	1	87	80
1	Lees	1	79	90
2	Parks	2	80	80

```
In [88]: # 현재 인덱스를 일반 열로 변경: index를 reset
ddf = df.reset_index(drop=True) #제거되는 index를 열로 포함 안함
ddf
```

```
Out[88]:
```

	Class	Eng	Mat
0	1	87	80
1	1	79	90
2	2	80	80

```
In [89]: # 현재 인덱스를 일반 열로 변경: index를 reset하고, 열로 추가(drop=False)
ddf = df.reset_index(drop=False, inplace=False) #drop=True는 제거되는 index를 열로 추가
ddf
```

```
Out[89]:
```

	Class	Eng	Mat
Kims	1	87	80
Lees	1	79	90
Parks	2	80	80

```
In [91]: # 현재 인덱스를 일반 열로 변경: index를 reset하고, 열로 추가(drop=False)
ddf = df.reset_index(drop=False, inplace=True) #drop=True는 제거되는 index를 열로 추가
ddf
```

```
Out[91]:
```

	index	Class	Eng	Mat
0	Kims	1	87	80
1	Lees	1	79	90
2	Parks	2	80	80

```
In [ ]:
```

```
In [94]: ## [Data Frame] 생성: index 지정 없이
import pandas as pd

df = pd.DataFrame({'Name' : ['Kims', 'Lees', 'Parks'],
                   'Class' : [1, 1, 2],
                   'Eng' : [87, 79, 80],
                   'Mat' : [80, 90, 80]},) #index 자동 부여
df
```

```
Out[94]:
```

	Name	Class	Eng	Mat
0	Kims	1	87	80
1	Lees	1	79	90
2	Parks	2	80	80

멀티 인덱스 중에 특정 열을 제거하는 방법

멀티 인덱스를 일반 컬럼으로 전환 후에 불필요한 컬럼 제거, 컬럼을 인덱스로 전환

```
In [96]: # 'Name' 열을 인덱스로 설정: 멀티 index 생성(append=True)
ddf = df.set_index('Name', append=True, drop=True, inplace=False) #'Name'으로 인덱스 생성
```

```
ddf
```

Out[96]:

		Class	Eng	Mat
--	--	-------	-----	-----

	Name			
0	Kims	1	87	80
1	Lees	1	79	90
2	Parks	2	80	80

reset_index(): 멀티 인덱스를 컬럼으로 전환

In [97]: *# 현재 인덱스를 일반 열로 변경: index를 reset하고, 열로 추가(drop=False)*
`ddf = ddf.reset_index(drop=False) # inplace=False는 자신은 안 바꾸고 변경 결과만 ddf`

Out[97]:

	level_0	Name	Class	Eng	Mat
--	---------	------	-------	-----	-----

0	0	Kims	1	87	80
1	1	Lees	1	79	90
2	2	Parks	2	80	80

drop(): 불필요한 컬럼 제거

In [77]: *## [Data Frame] 행, 열 제거*
`df = ddf.drop('level_0', axis = 'columns') # 컬럼 이름으로 컬럼 제거`
`df`

Out[77]:

	Name	Class	Eng	Mat
--	------	-------	-----	-----

0	Kims	1	87	80
1	Lees	1	79	90
2	Parks	2	80	80

In [78]: *## 데이터프레임의 복수열 추출*
`df = ddf[['Name', 'Class', 'Eng', 'Mat']]`
`df`

Out[78]:

	Name	Class	Eng	Mat
--	------	-------	-----	-----

0	Kims	1	87	80
1	Lees	1	79	90
2	Parks	2	80	80

.reindex()

: 데이터프레임이나 시리즈에서 인덱스나 컬럼 순서를 재정렬하거나 새로 지정할 때 사용

> new_index: 새로 지정할 인덱스 리스트

- > index : 행 인덱스 재정렬
- > columns : 열 이름 재정렬
- > fill_value : NaN 대신 채울 값
- > method : 보간 방식 (ffill, bfill 등)

```
In [8]: ## [Series] .reindex()로 행 순서 바꾸기
import pandas as pd

s = pd.Series([10, 20, 30], index=["a", "b", "c"])

# 인덱스 순서 바꾸기: + 없는 인덱스 추가
s2 = s.reindex(["c", "b", "a"])
print(s2)
```

```
c    30
b    20
a    10
dtype: int64
```

```
In [7]: ## [Series] .reindex()로 행 재구성하기
import pandas as pd

s = pd.Series([10, 20, 30], index=["a", "b", "c"])

# 인덱스 순서 바꾸기: + 없는 인덱스 추가
s2 = s.reindex(["c", "b", "d"])
print(s2)
```

```
c    30.0
b    20.0
d     NaN
dtype: float64
```

```
In [11]: ## [Dataframe] 인덱스와 컬럼 순서 모두 변경
df = pd.DataFrame({
    '국어': [90, 80, 70],
    '영어': [85, 95, 75]
}, index=["철수", "영희", "민수"])

# 인덱스와 컬럼 순서 모두 변경
df2 = df.reindex(index=["영희", "민수", "철수"],
                  columns=["영어", "국어"])
print(df2)
```

```
   영어  국어
영희   95   80
민수   75   70
철수   85   90
```

```
In [12]: ## [Dataframe] 인덱스와 컬럼 순서 모두 변경과 추가
df = pd.DataFrame({
    '국어': [90, 80, 70],
    '영어': [85, 95, 75]
}, index=["철수", "영희", "민수"])

# 인덱스와 컬럼 순서 모두 변경
df2 = df.reindex(index=["영희", "민수", "지수"],
                  columns=["영어", "수학"])
print(df2)
```

영 어 수 학
영 희 95.0 NaN
민 수 75.0 NaN
지 수 NaN NaN

(알아두면 좋아요) 리스트, 시리즈, 데이터 프레임 복사

- > 얇은 복사(shallow copy)와 깊은 복사(deep copy)가 있다.
- > 얇은 복사는 복사 대상을 공유하는 형태로 참조 방식
- > 깊은 복사는 복사 대상과 똑같은 데이터를 생성하여 항목을 복사하는 방식

```
In [79]: ## [Data Frame] 생성
import pandas as pd

df = pd.DataFrame({'Name' : ['Kims', 'Lees', 'Parks'],
                   'Class' : [1, 1, 2],
                   'Eng' : [87, 79, 80],
                   'Mat' : [80, 90, 80]}) #index 자동 부여

df
```

```
Out[79]:
```

	Name	Class	Eng	Mat
0	Kims	1	87	80
1	Lees	1	79	90
2	Parks	2	80	80

```
In [80]: ## 얇은 복사(shallow copy)
df_new = df # df_new가 df를 참조하는 형태로 복사
df['Mat'] = 100
df_new
```

```
Out[80]:
```

	Name	Class	Eng	Mat
0	Kims	1	87	100
1	Lees	1	79	100
2	Parks	2	80	100

```
In [81]: ## 깊은 복사(deep copy)
df_new = df[:] # df를 슬라이싱 해서 df_new로 복사하므로 두 프레임은 별개
df['Mat'] = 0
df_new
```

```
Out[81]:
```

	Name	Class	Eng	Mat
0	Kims	1	87	100
1	Lees	1	79	100
2	Parks	2	80	100

```
In [82]: ## 깊은 복사(deep copy)
df_new = df.copy()
df_new['Mat'] = 90
df_new
```

```
Out[82]:
```

	Name	Class	Eng	Mat
0	Kims	1	87	90
1	Lees	1	79	90
2	Parks	2	80	90

```
In [ ]:
```

데이터 프레임 데이터 조작

```
In [132... ## [Data Frame] 생성
import pandas as pd

df = pd.DataFrame({'Name' : ['Kims', 'Lees', 'Parks'],
                   'Class' : [1, 1, 2],
                   'Eng' : [87, 79, 80],
                   'Mat' : [80, 90, 80]},) #index 강제 부여

df
```

```
Out[132... 
```

	Name	Class	Eng	Mat
0	Kims	1	87	80
1	Lees	1	79	90
2	Parks	2	80	80

열 추가

```
In [133... ## [Data Frame] 조작 : 열 추가 or 열 추가
df['avg'] = (df['Eng'] + df['Mat']) / 2 # 'avg' 열을 만들어서 평균 값
df
```

```
Out[133... 
```

	Name	Class	Eng	Mat	avg
0	Kims	1	87	80	83.5
1	Lees	1	79	90	84.5
2	Parks	2	80	80	80.0

열 순서 변경

```
In [134... ## [Data Frame] 조작 : 열 순서 변경
df = df[['Class', 'Name', 'Eng', 'Mat', 'avg']] # 원하는 컬럼 순으로 재구성
df
```



```
Out[134...
```

	Class	Name	Eng	Mat	avg
0	1	Kims	87	80	83.5
1	1	Lees	79	90	84.5
2	2	Parks	80	80	80.0

```
In [135... ## [Data Frame] 조작 : 열 순서 변경
col_seq = ['Class', 'Name', 'Eng', 'Mat', 'avg']
df = df[col_seq] # 원하는 컬럼 순으로 재정렬
df
```

```
Out[135...
```

	Class	Name	Eng	Mat	avg
0	1	Kims	87	80	83.5
1	1	Lees	79	90	84.5
2	2	Parks	80	80	80.0

```
In [ ]:
```

데이터프레임 개별 항목 값 조작

> 반복문을 사용하여 개별 조작

> `.at[row_index, col_index] = value`

```
In [145... ## [Data Frame] 생성
import pandas as pd

df = pd.DataFrame({'Name' : ['Kims', 'Lees', 'Parks'],
                  'Class' : [1, 1, 2],
                  'Eng' : [87, 79, 80],
                  'Mat' : [80, 90, 80]},) #index 강제 부여
df
```

```
Out[145...
```

	Name	Class	Eng	Mat
0	Kims	1	87	80
1	Lees	1	79	90
2	Parks	2	80	80

```
In [146... ## [Data Frame] 조작 : 반복문 사용 접근
for index, row in df.iterrows():
    print(index, row['Name']) # .at[]은 행과 열로 특정값 검색
```

```
0 Kims
1 Lees
2 Parks
```

```
In [147... ## [Data Frame] 조작 : 반복문 사용 접근
for index, row in df.iterrows():
    print(index, df.at[index, 'Name']) # .at[]은 행과 열로 특정값 검색
```

0 Kims
1 Lees
2 Parks

```
In [148... ## [Data Frame] 조작 : 반복문 사용 조작
for index, row in df.iterrows():
    if df.at[index, 'Mat'] <= 90:
        df.at[index, 'Mat'] = df.at[index, 'Mat'] + 10
df
```

```
Out[148...      Name  Class  Eng  Mat
0    Kims      1   87   90
1    Lees      1   79  100
2    Parks      2   80   90
```

```
In [149... ## [Data Frame] 조작 : 열 값 일괄 적용 조작
df['Mat'] = df['Mat'] - 10
df
```

```
Out[149...      Name  Class  Eng  Mat
0    Kims      1   87   80
1    Lees      1   79   90
2    Parks      2   80   80
```

데이터 프레임 관련 함수 사용

```
In [150... ## [Data Frame] 생성
import pandas as pd

df = pd.DataFrame({'Name' : ['Kims', 'Lees', 'Parks'],
                   'Class' : [1, 1, 2],
                   'Eng' : [87, 79, 80],
                   'Mat' : [80, 90, 80]},) #index 강제 부여
df
```

```
Out[150...      Name  Class  Eng  Mat
0    Kims      1   87   80
1    Lees      1   79   90
2    Parks      2   80   80
```

```
In [151... ## [Data Frame] 관련 함수 : 통계 보기
sum(df['Eng'])
```

```
Out[151... 246
```

```
In [152... ## [Data Frame] 관련 함수 : 통계 보기
len(df) #행의 개수
```

Out[152...] 3

```
In [153...] ## [Data Frame] 관련 함수 : 통계 보기
avg_mat = sum(df['Mat']) / len(df)
print('Average of Math : ', avg_mat)
```

Average of Math : 83.33333333333333

```
In [154...] ## [Data Frame] 관련 함수 사용
x = df['Mat'];          # 'Mat' 열 값들을 시리즈로 추출
x.mean()                # 'Mat' 열 값들의 평균 구하기
```

Out[154...] np.float64(83.33333333333333)

```
In [155...] ## [Data Frame] 관련 함수 사용
x = df['Mat'];          # 'Mat' 열 값들을 시리즈로 추출
x.value_counts()        # 'Mat' 열 값들의 개 수 구하기
```

Out[155...] Mat
80 2
90 1
Name: count, dtype: int64

데이터 정렬

> .sort_values(by = 'avg')는 자신은 그대로 두고 정렬 결과를 반환

```
In [158...] ## [Data Frame] 관련 함수 : sorting된 Data Frame 생성
sdf = df.sort_values(by = 'Eng')      # 'avg' 컬럼 값 순으로 정렬
sdf
```

Out[158...]

	Name	Class	Eng	Mat
1	Lees	1	79	90
2	Parks	2	80	80
0	Kims	1	87	80

```
In [159...] ## .sort_values(by = 'Eng')는 자신은 그대로 두고 정렬 결과를 반환
df
```

Out[159...]

	Name	Class	Eng	Mat
0	Kims	1	87	80
1	Lees	1	79	90
2	Parks	2	80	80

```
In [160...] ## [Data Frame] 관련 함수 : sorting된 Data Frame 생성
sdf = df.sort_values(by = 'Eng', ascending=False)
sdf
```

Out[160...

	Name	Class	Eng	Mat
0	Kims	1	87	80
2	Parks	2	80	80
1	Lees	1	79	90

컬럼명 변경

> .rename()

In [161...

```
## [Data Frame] 관련 함수 : 컬럼명 변경
df = df.rename(columns = {'Mat' : 'Math'}) # 컬럼명 변경
df
```

Out[161...

	Name	Class	Eng	Math
0	Kims	1	87	80
1	Lees	1	79	90
2	Parks	2	80	80

행, 열 제거

> .drop()

In [167...

```
## [Data Frame] 관련 함수 : 행, 열 제거
ddf = df.drop('Class', axis = 'columns') # 컬럼 이름으로 컬럼 제거
ddf
```

Out[167...

	Name	Eng	Math
0	Kims	87	80
1	Lees	79	90
2	Parks	80	80

In [169...

```
## [Data Frame] 관련 함수 : 행, 열 제거
ddf = df.drop('Class', axis = 1) # 행 index로 열(axis=1) 제거
ddf
```

Out[169...

	Name	Eng	Math
0	Kims	87	80
1	Lees	79	90
2	Parks	80	80

In [170...

```
## [Data Frame] 관련 함수 : 행, 열 제거
ddf = df.drop(1, axis = 0) # 행 index로 행(axis=0) 제거
ddf
```

Out[170...

	Name	Class	Eng	Math
0	Kims	1	87	80
2	Parks	2	80	80

In [171... `## [Data Frame] 관련 함수 : 행, 열 제거`
`ddf = ddf.drop(columns=['Eng', 'Math'])` # 복수개 컬럼 이름으로 컬럼 제거
`ddf`

Out[171...

	Name	Class
0	Kims	1
2	Parks	2

행 일부만 출력

> `.head()`
 > `.tail()`

In [172... `## [Data Frame] 관련 함수 : 일부만 출력`
`df.head(2)` # 데이터 행 상위 일부만 출력 (숫자 생략하면 알아서)

Out[172...

	Name	Class	Eng	Math
0	Kims	1	87	80
1	Lees	1	79	90

In [173... `## [Data Frame] 관련 함수 : 일부만 출력`
`df.tail(1)` # 데이터 행 하위 일부만 출력 (숫자 생략하면 알아서)

Out[173...

	Name	Class	Eng	Math
2	Parks	2	80	80

행과 열의 수 출력

> `.shape()`

In [174... `## [Data Frame] 관련 함수 : 행, 열의 수 출력`
`df.shape`

Out[174... (3, 4)

프레임 속성 출력

> `.info()`

In [175... `## [Data Frame] 관련 함수 : 데이터 프레임 속성 출력`
`df.info()`

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 3 entries, 0 to 2
Data columns (total 4 columns):
#   Column  Non-Null Count  Dtype
---  -
0    Name      3 non-null      object
1    Class     3 non-null      int64
2    Eng       3 non-null      int64
3    Math      3 non-null      int64
dtypes: int64(3), object(1)
memory usage: 228.0+ bytes
```

프레임 통계 출력

```
> .describe()
```

```
In [176... ## [Data Frame] 관련 함수 : 통계 보기
df.describe()
```

```
Out[176...
      Class      Eng      Math
count  3.000000  3.000000  3.000000
mean    1.333333  82.000000  83.333333
std     0.577350  4.358899  5.773503
min     1.000000  79.000000  80.000000
25%     1.000000  79.500000  80.000000
50%     1.000000  80.000000  80.000000
75%     1.500000  83.500000  85.000000
max     2.000000  87.000000  90.000000
```

```
In [ ]:
```

□ Excel이나 csv 타입 파일로부터 데이터 프레임 구성하기

> import pandas 를 통해 read_excel() 또는 read_csv()를 사용한다.

>> 데이터에 한글이나 특수 문자가 포함되어있는경우 encoding='UTF-8' 적용

```
In [177... ## [Data Frame] Excel 파일 >> 데이터 프레임 구축
import pandas as pd

df_exam = pd.read_excel('excel_exam.xlsx', header=None) #header가 없을 때, 데이터
df_exam = pd.read_excel('excel_exam.xlsx')             #header가 있을 때, 첫
df_exam.head()
```

Out[177...

	id	nclass	math	english	science
--	----	--------	------	---------	---------

0	1	1	50	98	50
1	2	1	60	97	60
2	3	1	45	86	78
3	4	1	30	98	58
4	5	2	25	80	65

In [178...

```
## [Data Frame] Excel 파일 >> 데이터 프레임 구축
import pandas as pd

df_exam = pd.read_excel('excel_exam.xlsx', sheet_name='Sheet2', header=None) #
#header가 없을 때, 데이터 프레임에 열 index가 자동 부여됨
df_exam.head()
```

Out[178...

	0	1	2	3	4
--	---	---	---	---	---

0	1	1	50	98	50
1	2	1	60	97	60
2	3	1	45	86	78
3	4	1	30	98	58
4	5	2	25	80	65

In [179...

```
## [Data Frame] 데이터 프레임 >> csv 파일로 저장
import pandas as pd

df_exam = pd.read_excel('excel_exam.xlsx') #header가 있을 때, 첫 행이 데이터 프
df_exam.to_csv("exam_csv.csv")
df_exam.to_csv("exam_csv.csv", index=False) #index는 빼고 저장

df_exam = pd.read_csv('exam_csv.csv') #header가 있을 때, 첫 행이 데이터 프레임
df_exam.head()
```

Out[179...

	id	nclass	math	english	science
--	----	--------	------	---------	---------

0	1	1	50	98	50
1	2	1	60	97	60
2	3	1	45	86	78
3	4	1	30	98	58
4	5	2	25	80	65

In []:

[과제] 데이터 프레임 조작하기

1. Excel > Data Frame 변환

> 'excel_exam.xlsx' 파일을 읽어서 데이터 프레임 구성

2. 'total'과 'avg' 열 변수 추가

> 'total'은 'math', 'english', 'science'의 합

> 'avg'는 'math', 'english', 'science'의 평균

3. 'pass' 열 변수를 추가

> 'pass'를 'None' 값으로 초기화

4. 'pass' 값을 변경

> 'pass'는 'avg' 값이 60 이상이면 'P', 아니면 'F'로 값 변경

5. 결과 행이 선택된 'df_pass' 데이터 프레임 생성

> 'pass'가 'P' 행들만 선택

> 선택된 행 중에 'id', 'nclass', 'math', 'english', 'science', 'avg' 열만 추출해서 구성

> 열 순서를 'nclass', 'id', 'math', 'english', 'science', 'avg' 순으로 구성

6. Data Frame > csv 파일로 변환

> 파일로 변환하기 전에 'df_pass' 데이터 프레임을 'avg' 내림차 순으로 정렬

> 데이터 프레임을 'excel_exam_pass.csv' 파일로 저장 (단, index는 빼고)

7. Data Frame > csv 파일로 변환

> 'excel_exam_pass.csv' 파일을 데이터 프레임으로 읽어서, 출력 확인

In []: